

A SALES ENGINEER'S GUIDE

The Busy Town

Understanding the Model Context Protocol through story

Relay floating

How messages move between models, clients, tools, and data—
and why the piece everyone forgets is the one that matters most.

WELCOME

Meet Relay

Hello! I'm **Relay**—a small floating courier robot. I carry messages between the buildings of a busy little town. Every day, requests arrive from citizens and responses go out from Town Hall. My job is to make sure every message reaches exactly where it needs to go.

Relay the courier robot

This town runs on a system you might recognize. In the world of software, it's called the **Model Context Protocol**—MCP for short. But before we get technical, let me show you around.

In this guide, you'll learn:

- How **Town Hall** makes decisions—and why it can never leave the building
- Why the **Courier Station** is the most important building in town
- What the **Tool Workshop** builds and how tools get used
- Where the **Distant Warehouse** keeps its records—and why real data matters
- How a single request travels through the entire town and back

Who is this for?

Sales engineers and technical professionals who want a clear, jargon-light introduction to MCP architecture—and a confident way to explain it to enterprise clients.



OVERVIEW

The Town Map

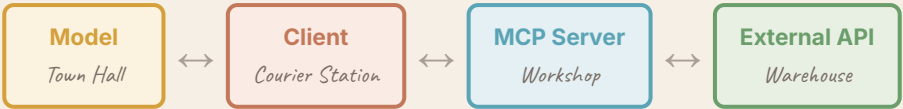
Every building has a role. Every road carries a message. Let me show you how it all connects.

Bird's eye view of The Busy Town with labeled buildings

The Busy Town – where every message has a journey

- Courier Station — The Client
- Town Hall — The Model
- Tool Workshop — The MCP Server
- Distant Warehouse — The External API

THE REAL ARCHITECTURE



SECTION ONE

The Courier Station

Of all the buildings in town, the Courier Station is the easiest to overlook. It isn't grand like Town Hall. It doesn't have the fascinating tools of the Workshop. Nobody writes songs about the Courier Station.

But without it, **nothing** moves.

Every morning, the couriers line up for their routes. Requests arrive from citizens. Each one gets wrapped in a neat envelope, stamped with a destination, and dispatched. The couriers carry questions to Town Hall, carry decisions back, ferry tool requests to the Workshop, and deliver final answers to the citizens.

The busy Courier Station with animal workers carrying messages

The Courier Station at peak hour — every message passes through here

If you removed any other building, the town would struggle. If you removed the Courier Station, the town would **stop**.

I know this building well. It's where I was built. People forget the courier because it isn't glamorous—but the courier is the reason anything works at all.

The Client Application

● WHAT THE CLIENT ACTUALLY DOES

In MCP, the **Client** is the application layer that sits between the user and everything else. It is responsible for:

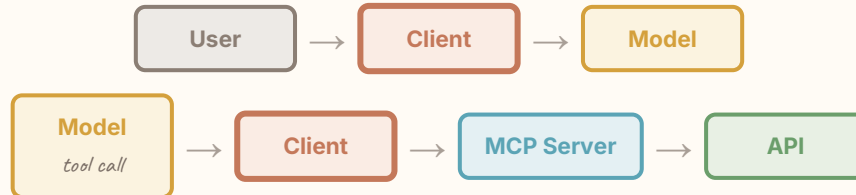
- **Connecting** to one or more MCP servers and discovering available tools
- **Sending prompts** to the model (the LLM)
- **Receiving tool-call requests** from the model when it decides a tool is needed
- **Executing** those tool calls against the appropriate MCP server
- **Returning results** to the model so it can formulate a final response
- **Delivering** the response back to the user

Many teams focus on the model's intelligence or the server's tools. But the client is the **orchestrator**. Without it, the model cannot reach the tools, and the tools cannot reach the data.

The Key Insight

The model is the brain. The client is the nervous system. Intelligence without connectivity is just potential.

Request flow through the Client



The Client (bold border) appears in every path. Nothing bypasses it.

SECTION TWO

Town Hall

Town Hall sits at the center of everything. Inside, the Council—wise, thoughtful, deeply knowledgeable—listens to every request and decides what needs to happen.

But here's the thing about the Council: **they never leave the building**. They can reason, analyze, and decide. They can specify exactly which tool is needed and how it should be used. But they cannot walk to the Workshop themselves. They cannot fetch records from the Warehouse. They write instructions on scrolls and hand them to the courier.

Grand Town Hall with the wise Council deliberating inside

The Council is brilliant – but bound to its chamber

The Council is brilliant but bound. That is by design.

● BEHIND THE SCENES — THE MODEL

The **Model** (typically an LLM such as Claude) processes input, reasons about it, and determines whether to respond directly or call a tool. When it decides a tool is needed, it emits a structured **tool-call request**—specifying the tool name and arguments in JSON.

But the model never executes the call itself. It cannot make HTTP requests, query databases, or access files. **That's the client's job.**

SECTION THREE

The Tool Workshop

The Tool Workshop is where clever devices are built and maintained. Each tool has a clear name, a defined purpose, and precise instructions for use: the Knowledge Search Bell, the Document Stamp, the Archive Key.

When a courier arrives with a tool request from Town Hall, the Workshop doesn't question why. It takes the request, executes the tool, and hands the result back. The Workshop doesn't know what the Council plans to do with the answer. It doesn't need to. **Each building has its own responsibility.**

Tool Workshop filled with labeled tools and busy craftworkers

Every tool has a name, a purpose, and an instruction card

● BEHIND THE SCENES — THE MCP SERVER

An **MCP Server** exposes tools to the client. Each tool is defined with:

- A **name** (e.g., `search_knowledge_base`)
- A **description** (what the tool does, in natural language)
- An **input schema** (what arguments it accepts, defined in JSON Schema)

The server receives tool calls from the client, executes them against backends (databases, APIs, file systems), and returns structured results.

SECTION FOUR

The Distant Warehouse

Outside town, past the bridge and over the hill, sits the Warehouse. It's vast—rows upon rows of records, documents, and verified data.

The Warehouse doesn't care who's asking. It responds to anyone who rings the right bell with the right request. It's fast, impersonal, and reliable. But you can't just walk up and ask it a question in plain language. You need to use the proper tool—the Knowledge Search Bell—and format your request correctly.

The Distant Warehouse on a hill, filled with records

The Warehouse holds the truth – verified, current, and real

● BEHIND THE SCENES — EXTERNAL APIS

External APIs are the ultimate data sources: search engines, databases, CRMs, compliance systems, knowledge bases. The MCP server wraps these APIs in tool definitions so the model can use them without knowing the underlying endpoints, authentication, or response formats.

The data in these systems is **real, current, and authoritative**—which is exactly why connecting to it matters so much. The next page explains why.

SECTION FIVE

Why the Warehouse Matters

There was a time, early in the town's history, when Town Hall tried to answer every question from memory alone.

A citizen would ask about the latest compliance requirements. The Council would think deeply, recall what they knew, and offer an answer. The answers were always eloquent. They sounded authoritative. But sometimes—more often than anyone was comfortable with—**they were wrong**.

Not deliberately wrong. The Council simply didn't know what it didn't know. Regulations had changed. New records had been filed. The world had moved on, but the Council's memory hadn't.

That's when the town built the Warehouse. Now, when a citizen asks about compliance, the Council doesn't guess. It sends a request—through the courier, through the Workshop, to the Warehouse—and gets **the actual answer**.

The Council comparing scrolls from the Warehouse against memory

For the businesses in town, this changed everything.

● BEHIND THE SCENES — GROUNDING IN REAL DATA

Large Language Models can **hallucinate**—generate responses that sound correct but aren't. For enterprise applications, this is not just an inconvenience. It's a liability.

Retrieval-Augmented Generation (RAG) solves this by grounding responses in real data. MCP provides the standardized protocol to connect models to authoritative sources. When grounded, you get:

- **Accuracy** — answers reflect current reality, not stale training data
- **Auditability** — every claim traces to a verifiable source
- **Trust** — stakeholders can verify the information
- **Compliance** — critical for regulated industries

Without grounding, AI generates plausible fiction. With grounding, it becomes a **reliable business tool**.

SECTION SIX

A Request's Journey

Let me walk you through a typical day. Follow me—I'll carry this one myself.

A citizen approaches the Courier Station with a question: "What are the latest compliance requirements for our industry?"

Relay carrying a message through the town streets

Every request takes the same path through town.

- 1 The citizen hands the request to the **Courier Station**.
- 2 The courier wraps it in a clean envelope and carries it to **Town Hall**.
- 3 **The Council** reads the question. They don't have the latest data, but they know exactly what to do: "Use the Knowledge Search Bell. Search for: compliance requirements, current year."
- 4 The courier carries this tool request to the **Tool Workshop**.

Notice: the Council decided which tool to use, but the courier is the one who actually carries the request. The Council never leaves its chamber.

The Journey Home

- 5 The **Workshop** takes the scroll, rings the Knowledge Search Bell, and sends the signal to the **Distant Warehouse**.
- 6 The **Warehouse** returns three matching compliance documents—real, verified, current.
- 7 The **Workshop** hands the results to the courier, who carries them back to **Town Hall**.
- 8 **The Council** reads the real data, synthesizes a clear, grounded answer, and writes a response.
- 9 The courier delivers the final answer to the citizen. **Accurate. Sourced. Trustworthy.**

● THE COMPLETE FLOW



Results return along the same path: API → MCP → Client → Model → Client → User

Every arrow passes through the Client. The model decides. The client carries. The server executes. The API responds. And the client carries it all home.

Nine steps. Four buildings. One courier. Every answer grounded in real data from the Warehouse.

SECTION SEVEN

When the Courier Rests

One Tuesday, the Courier Station closed early. Maybe it was a holiday. Maybe there was a broken wheel on the main cart.

Town Hall had brilliant ideas but no way to share them. The Workshop had every tool polished and ready, but no one came. The Warehouse had fresh data waiting, but no bell rang.

The citizens waited. And waited.

The quiet, empty town with idle buildings and waiting citizens

Without the courier, the town is a collection of lonely buildings

This is What Happens When the Client Goes Down

The model still works. The tools still work. The APIs still respond. But **nothing connects**. The entire flow depends on the one piece of infrastructure that everyone takes for granted.

● BEHIND THE SCENES — CLIENT RELIABILITY

The client is the **single point of orchestration**. If it crashes, times out, or misconfigures its connections, the entire MCP flow stops—even though every other component is healthy.

This is why client reliability, error handling, connection management, and graceful degradation matter so much in production. The client isn't glamorous, but it is **critical infrastructure**.

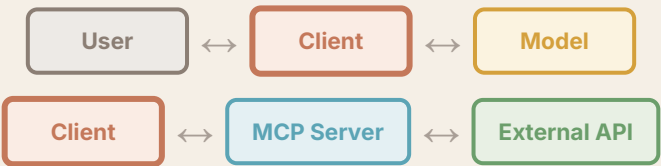
PUTTING IT ALL TOGETHER

The Full Picture

Now you've seen the whole town. Here's how the story maps to the real system:

In the Town	In the System	Role
Courier Station	Client Application	Orchestrates all message flow
Town Hall / Council	LLM (Model)	Reasons and decides which tools to call
Tool Workshop	MCP Server	Exposes and executes tools
Distant Warehouse	External API / Database	Stores real, authoritative data
Citizens	End Users	Ask questions, receive answers
Scrolls & Envelopes	JSON Messages	Structured data in transit
Knowledge Search Bell	Tool Definition	Named, described, schema-defined capability

● THE COMPLETE ARCHITECTURE



The model chooses. The client carries. The server executes. The data grounds the answer.

Three Things to Remember

1. The model is powerful but isolated. It reasons and decides—it never acts directly.
2. The client is the unsung hero. Every message, every tool call, every response flows through it.
3. Real data matters. Grounding responses in authoritative sources is what makes AI trustworthy for business.

Relay waving goodbye

**Every message matters.
Every courier counts.**

Thank you for visiting the Busy Town.

– Relay